

Python Environments Cheat Sheet: venv vs Conda

Purpose of Python Environments

Python virtual environments allow you to isolate project dependencies so that different projects can use different versions of Python and third-party packages without conflicts.

venv Overview

- Built into Python (standard library).
- Lightweight and simple.
- Uses pip for package management.
- Ideal for most general-purpose Python projects.

Conda Overview

- Separate tool (Anaconda or Miniconda).
- Manages both Python and non-Python dependencies.
- Includes its own package manager.
- Common in data science and scientific computing.

Creating Environments

venv:

```
python -m venv myenv
```

Conda:

```
conda create -n myenv python=3.11
```

Activating Environments

venv:

Windows:

```
myenv\Scripts\activate
```

macOS/Linux:

```
source myenv/bin/activate
```

Conda:

```
conda activate myenv
```

Installing Packages

venv:

```
pip install requests
```

Conda:

```
conda install numpy
```

Key Differences

- venv is included with Python; Conda must be installed separately.
- venv relies on pip; Conda has its own package resolver.
- Conda can install compiled system libraries (e.g., BLAS, CUDA).
- venv environments are smaller and faster to create.

When to Use Each

Use venv when:

- You want a simple, lightweight setup.
- You are developing typical Python applications.

Use Conda when:

- You need complex scientific libraries.
- You want Python version management included.
- You are working in data science or machine learning.

Best Practices

- Use one environment per project.
- Never install packages globally unless necessary.
- Document dependencies using requirements.txt or conda env export.
- Be consistent within a team—don't mix venv and Conda arbitrarily.